

-> Experiment done in lab: Number Plate Recognition Backend System

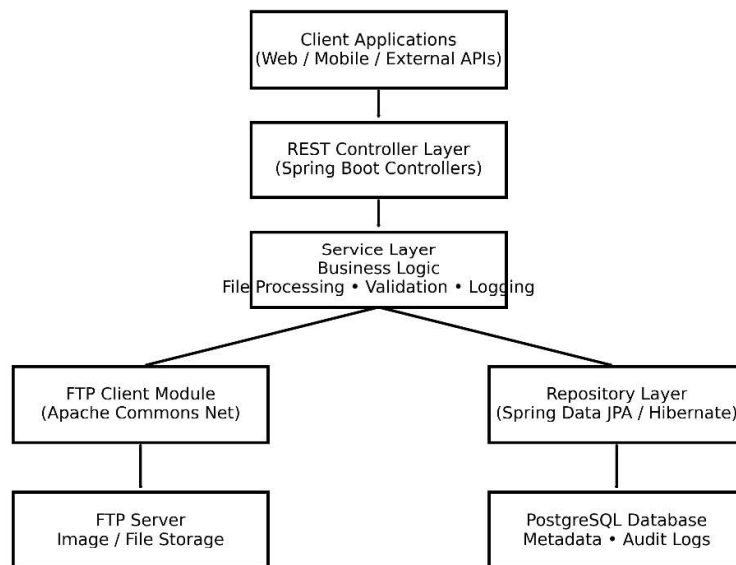
1. Introduction

The project presents the development of a backend microservice for number plate recognition and file management, implemented using the Spring Boot framework. The backend system is designed to support operations such as image upload, retrieval, and storage, while also maintaining an audit trail of file activities in a database.

The system integrates multiple technologies including REST APIs, FTP communication, and relational database management to create a scalable and maintainable architecture. The backend service acts as a bridge between client applications and the storage system where images and processed results are stored.

The primary objective of the project is to provide a robust and modular backend infrastructure capable of handling number plate image files and managing them efficiently while enabling seamless communication between different components of the application.

2. System Architecture



The system follows a multi-layered architecture, which ensures separation of concerns and improves maintainability. Each layer is responsible for a specific function in the application.

The architecture consists of the following layers:

2.1 Controller Layer

The controller layer handles incoming HTTP requests from client applications. It exposes RESTful APIs that allow clients to:

- Upload number plate images
- Retrieve stored images
- Request archived folders
- Access processed files

This layer serves as the entry point of the application and forwards the requests to the service layer for further processing.

2.2 Service Layer

The service layer contains the core business logic of the system. It acts as an intermediary between the controllers and the data access layer.

Responsibilities include:

- Managing file upload operations
- Handling image processing workflow
- Communicating with the FTP server
- Logging file activities
- Ensuring validation and error handling

By separating business logic from request handling, the system becomes more modular and scalable.

2.3 Data Access Layer

The data access layer manages interactions with the PostgreSQL database using Spring Data JPA and Hibernate.

Key responsibilities include:

- Storing file metadata
- Maintaining audit logs
- Retrieving stored records
- Managing database transactions

Hibernates automatic schema generation feature (ddl-auto=update) simplifies database setup by automatically creating required tables.

3. Technologies Used

The backend system integrates several technologies to achieve efficient file handling and database management.

Spring Boot

Spring Boot serves as the core backend framework. It simplifies the development process by providing built-in configurations, dependency injection, and embedded server support.

Advantages include:

- Rapid development
- Simplified configuration
- Scalable architecture
- REST API support

PostgreSQL Database

PostgreSQL is used as the relational database system to store application data such as file metadata and logs. It provides:

- Reliable storage
- High performance
- Transaction support
- Scalability

Spring Data JPA and Hibernate

Spring Data JPA simplifies database operations by allowing developers to interact with the database through repository interfaces instead of writing complex SQL queries.

Hibernate acts as the Object Relational Mapping (ORM) tool that converts Java objects into database records.

Apache Commons Net (FTPClient)

The system uses Apache Commons Net library to handle FTP communication. The FTPClient class enables:

- Connecting to the FTP server
- Uploading image files
- Downloading stored files
- Managing directories

This allows the backend to store images remotely and manage them efficiently.

4. File Management Workflow

The system performs several operations to manage number plate images effectively.

Image Upload

1. A client sends an image through a REST API request.
2. The controller receives the request.

3. The service layer processes the file.
4. The file is uploaded to the FTP server.
5. Metadata about the file is stored in the PostgreSQL database.

Image Retrieval

1. A client requests a stored image.
2. The system retrieves the corresponding record from the database.
3. The FTP server fetches the image.
4. The image is returned to the client.

Folder Archiving

The system supports dynamic archiving of folders. This feature allows multiple files to be compressed and retrieved as a single archive, making data transfer more efficient.

5. Audit Trail and Logging

A key feature of the backend system is maintaining an audit trail of all file-related operations.

The database stores details such as:

- File name
- Upload time
- File path
- Operation type
- User request information

This ensures traceability and accountability within the system.

Audit logs are useful for:

- Debugging errors
- Monitoring system activity
- Ensuring data integrity
- Tracking file usage

6. Advantages of the System

The backend system offers several benefits:

1. Scalability - The modular architecture allows the system to scale easily as the number of users and files increases.

2. Maintainability - Separation of layers ensures that updates to one component do not affect others.
3. Efficient File Management - FTP integration enables reliable storage and retrieval of image files.
4. Database Integration - PostgreSQL ensures secure and structured storage of metadata and logs.
5. REST API Support - The backend can easily integrate with web applications, mobile apps, or other services.

8. Conclusion

The developed backend system provides a reliable and scalable infrastructure for managing number plate images. By combining Spring Boot, PostgreSQL, FTP integration, and REST APIs, the system efficiently handles file storage, retrieval, and logging operations.

The layered architecture improves maintainability and allows the system to integrate easily with frontend applications or machine learning modules used for number plate recognition.

Overall, the project demonstrates how modern backend technologies can be combined to build a robust microservice capable of handling real-world file processing workflows.